LogitFactory: Training-Free Vocabulary Pruning for Large-Language-Model Inference

Myeong Jun Jo
ANIMA Research, Independent Researcher, South Korea
ORCID: 0009-0006-9540-4666

Abstract
Autoregressive large language model (LLM) inference multiplies the final hidden state against the entire output embedding matrix at every generation step, producing a logit vector of size V of which over 99.99% of entries are discarded. For a model with $V = 256{,}000$ and hidden dimension $d = 12{,}288$, this is approximately 3×10^9 FLOPs per token. The shape of this computation has not changed since the original Transformer. This paper presents LogitFactory, a training-free method that prunes the vocabulary before logit computation using PCA-based cluster alignment and dual-directional filtering. The method achieves 96–100% top-1 accuracy on GPT-2 variants while reducing logit FLOPs by 70–99%. Two implementation choices are essential. First, cosine similarity between the hidden state and vocabulary cluster centroids yields 0% top-1 accuracy on all tested configurations; replacing this with dot-product comparison on PCA-clustered centroids in the original embedding space recovers near-full accuracy. Second, a sample-first execution order runs at $0.09\times$ full-matmul speed; inverting to cluster-first execution raises this to $0.86\times$ ($9.6\times$ process-level improvement). Using cluster-first execution in pure PyTorch, the method reaches $1.09\times$ wall-clock speedup at $V = 256{,}000, d = 12{,}288$ (GPT-4 class), $1.97\times$ at $V = 500{,}000$, and $2.59\times$ at $V = 500{,}000, d = 16{,}384$. The speedup ratio grows monotonically with V and d.

Keywords: logit factoring, vocabulary pruning, inference acceleration, space alignment, cluster-first execution

1 Introduction

1.1 The Unquestioned Multiplication

Every token an LLM generates requires a single operation: multiplying the final hidden state $\mathbf{h} \in \mathbb{R}^d$ against the entire output embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$ to produce a logit vector over the full vocabulary V.

For a model with $d = 4{,}096$ and $V = 256{,}000$, this is approximately 1.05×10^9 multiply-add operations per token. The model then applies softmax, selects one token (or samples from the top few), and discards the remaining $255{,}999$ scores. This process repeats at every generation step, in every model, at every inference provider. The shape of this computation has not changed since the original Transformer [Vaswani et al., 2017].

Over 99.99% of the computed scores are unused. Yet no published work has proposed pruning the vocabulary *before* logit computation.

1.2 Why the Problem Stayed Open

Section 4 documents the specific reason the problem went unsolved: the most obvious attempt fails completely. The intuitive approach—cosine similarity between the hidden state and vocabulary cluster centroids—produces 0.0% top-1 accuracy on GPT-2 across every configuration tested. A researcher encountering this result would reasonably conclude that vocabulary pruning is incompatible with the Transformer architecture. The failure provides no gradient toward a fix.

This paper identifies two barriers that previously blocked progress:

1. **Geometric misalignment.** The hidden state and embedding vectors occupy spaces with compatible dot-product structure but incompatible angular structure. Dot product on PCA-clustered centroids computed in the original embedding space converts 0% accuracy into 100%.
2. **Execution order.** A functionally correct implementation executed sample-first is slower than the full matmul it replaces. Inverting to cluster-first execution delivers speedups that scale with V and d.

1.3 Contributions

1. A training-free method (LogitFactory) that prunes the vocabulary before logit computation using two complementary selectors operating in opposition.
2. Identification of geometric misalignment as the barrier to vocabulary pruning, resolved by dot-product alignment on PCA-clustered centroids. Demonstration of $0\% \rightarrow 100\%$ recovery on three GPT-2 variants.
3. Identification of execution order as a second, independent barrier. A correct implementation executed sample-first runs at $0.09\times$ full matmul; cluster-first execution yields $0.86\times$ at the same fidelity.
4. A scaling benchmark showing that the speedup grows monotonically with V and d, reaching $1.09\times$ at $V = 256{,}000, d = 12{,}288$ (GPT-4 class), $1.97\times$ at $V = 500{,}000$, and $2.59\times$ at $V = 500{,}000, d = 16{,}384$ in pure PyTorch.

2 Related Work

2.1 Post-Computation Filtering

Top-K and nucleus (top-p) sampling [Holtzman et al., 2020] filter the logit distribution after full computation. They improve generation diversity but provide zero computational savings: every score is still computed.

2.2 Static Vocabulary Partitioning

Adaptive Softmax [Grave et al., 2017] partitions the vocabulary by frequency at training time, computing full-dimensional scores only for high-frequency tokens. The partition is fixed and context-independent. Hierarchical Softmax [Morin and Bengio, 2005] organizes the vocabulary as a binary tree, reducing computation to $O(\log V)$ but requiring sequential traversal incompatible with GPU parallelism. Both methods require retraining. Neither adapts to context.

2.3 Speculative Decoding

Speculative decoding [Leviathan et al., 2023] reduces the number of autoregressive steps using a small draft model, but does not reduce the computation within each step. The full vocabulary logit computation is still performed at every verification step.

2.4 Token Compression

Token pruning for vision transformers [Rao et al., 2021] and multimodal LLMs [Chen et al., 2024] reduces the number of input tokens processed. This is orthogonal to the work presented here, which reduces the output vocabulary scored at the final layer.

2.5 Position of This Work

No prior work prunes the output vocabulary dynamically, per token, before logit computation. Table 1 summarizes the distinction.

Table 1: Comparison with existing approaches.

	When	Context-adaptive	Retrain	Saving
Top-K/Top-p	After	No	No	0%
Adaptive Softmax	Fixed	No	Yes	50–70%
Hierarchical Softmax	Fixed	No	Yes	60–80%
Speculative Decoding	N/A	Yes	Separate	40–60%
LogitFactory (ours)	**Before**	**Yes**	**No**	**70–99%**

3 Method

3.1 Overview

Given a hidden state $\mathbf{h}$ at the final layer, the standard approach computes $\boldsymbol{\ell} = \mathbf{h}\mathbf{W}^\top \in \mathbb{R}^V$. LogitFactory replaces this with a four-stage pipeline:

1. **Space Alignment** (Sec. 3.3): Compare $\mathbf{h}$ against vocabulary clusters using dot product on PCA-clustered centroids in original space.
2. **Reverse Rotary** (Sec. 3.4): Eliminate clusters that are certainly irrelevant (recall-oriented).
3. **Forward Rotary** (Sec. 3.5): Select clusters that are certainly relevant (precision-oriented).
4. **Precise Computation** (Sec. 3.7): Compute exact logits only for the intersection, using cluster-first execution (Sec. 3.9).

3.2 Vocabulary Clustering (Offline)

The output embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$ is clustered offline via PCA-reduced K-means. PCA reduces $\mathbf{W}$ to k dimensions ($k = 64$ in the experiments reported here), then clusters the reduced representations into C groups. The cluster centroids are computed in the original d-dimensional space:

$$\mathbf{c}_j = \frac{1}{|G_j|} \sum_{i \in G_j} \mathbf{w}_i \tag{1}$$

where G_j is the set of token indices assigned to cluster j. This is performed once at deployment.

3.3 Space Alignment

The hidden state $\mathbf{h}$ and the embedding vectors $\mathbf{w}_i$ occupy spaces with different geometric structure. Direct cosine similarity between $\mathbf{h}$ and the centroids $\mathbf{c}_j$ produces meaningless rankings—a fact documented empirically in Section 4.

The resolution is to compare $\mathbf{h}$ and $\mathbf{c}_j$ via dot product rather than cosine similarity, after clustering $\mathbf{W}$ in PCA-reduced space. The PCA basis provides an implicit orthogonal projection that aligns the comparison geometry. No additional learned parameters are required: the PCA components of $\mathbf{W}$ are computed offline, and the centroids in original space serve as the comparison targets.

The entire method is training-free: no gradient updates, no fine-tuning, no paired data. The projection is derived entirely from the model's own embedding matrix.

3.4 Reverse Rotary (Elimination)

The reverse rotary computes similarity scores between the hidden state and all C centroids, then eliminates the bottom $P\%$:

$$s_j = \mathbf{h} \cdot \mathbf{c}_j, \quad j = 1, \ldots, C \tag{2}$$

$$\mathcal{R} = \{j : s_j \in \text{bottom-}P\%(\{s_1, \ldots, s_C\})\} \tag{3}$$

This stage is conservative (high recall): it removes only clusters that are clearly irrelevant, ensuring the correct token's cluster is retained.

3.5 Forward Rotary (Selection)

From the surviving clusters $\{1, \ldots, C\} \setminus \mathcal{R}$, the forward rotary selects the top M by score:

$$\mathcal{F} = \text{top-}M\big(\{s_j : j \notin \mathcal{R}\}\big) \tag{4}$$

This stage is aggressive (high precision): it retains only the most relevant clusters.

3.6 Why Two Rotaries

A single selector faces a precision-recall tradeoff. Reverse-only (elimination) leaves too many candidates; forward-only (selection) risks missing the correct cluster. Composing elimination-then-selection achieves a balance that neither stage reaches independently. Section 5.5 reports the ablation.

3.7 Precise Logit Computation

The final candidate set is the union of tokens in the selected clusters:

$$\mathcal{S} = \bigcup_{j \in \mathcal{F}} G_j \tag{5}$$

Exact logits are computed only for $\mathcal{S}$:

$$\ell_i = \mathbf{h} \cdot \mathbf{w}_i, \quad i \in \mathcal{S} \tag{6}$$

followed by softmax and sampling over $\mathcal{S}$. Typical $|\mathcal{S}|$ is 1–10% of V.

3.8 Fallback Mechanism

If $\max(\text{softmax}(\boldsymbol{\ell}_{\mathcal{S}})) < \theta_{\text{fallback}}$, the system falls back to full vocabulary computation, guaranteeing that generation quality is never worse than the baseline.

3.9 Cluster-First Execution

A functionally correct implementation is not necessarily a fast one. The intuitive sample-first pattern—for each sample n, gather its selected embedding rows $\mathbf{W}[\mathcal{S}_n]$ and compute $\mathbf{h}_n \cdot \mathbf{W}[\mathcal{S}_n]^\top$—runs at $0.09\times$ the speed of full matmul on Qwen-72B scale. Per-sample gather prevents tensor cores from amortizing setup cost across a batch.

The fix inverts the loop:

- **Offline**: permute $\mathbf{W}$ so that tokens belonging to cluster j occupy a contiguous slice $\mathbf{W}_{\text{sorted}}[s_j : e_j]$.
- **Online**: iterate over clusters, not samples. For each cluster j, collect the samples that selected it, slice the matrix contiguously, and perform one batched matmul.

This single restructuring raises the speed ratio from $0.09\times$ to $0.86\times$—a $9.6\times$ process-level improvement with no change to the algorithm's mathematics.

4 The Zero-Percent Wall

Before presenting positive results, this section reports the failure that preceded them.

The initial implementation clustered $\mathbf{W}$ directly (without PCA), L_2-normalized all vectors, and used cosine similarity to rank clusters. On GPT-2 (50,257 vocabulary, 768 dimensions), this achieved 0.0% top-1 accuracy—across every combination of $C \in \{50, 100, 200, 300, 500\}$ and $M \in \{1, 2, 3, 5, 7, 10, 15, 20, 30\}$. On 20 manually selected prompts spanning 10 domains, the predicted token matched the ground truth zero times.

The hidden state $\mathbf{h}$ and the embedding row $\mathbf{w}_i$ are related by $\ell_i = \mathbf{h} \cdot \mathbf{w}_i$, but their angular relationships diverge from their dot products: the cosine similarity between $\mathbf{h}$ and $\mathbf{c}_j$ does not predict the magnitude of $\mathbf{h} \cdot \mathbf{c}_j$. Normalizing both vectors destroys the scale information that carries the actual logit signal.

The fix, described in Section 3.3, is to use dot product on PCA-clustered centroids in original space. The difference is not incremental: it is 0% versus 96–100%.

5 Experiments

Four experiment families are reported: (1) accuracy on three GPT-2 scales, (2) the space-alignment ablation behind the $0\% \rightarrow 100\%$ recovery, (3) the cluster-first redesign, and (4) scaling-law speedups up to $V = 500{,}000$, $d = 16{,}384$.

5.1 Setup

Small-vocabulary accuracy experiments ran on Kaggle (Tesla T4, 16 GB). Large-scale speed experiments ran on Modal A100 80 GB. All software is PyTorch; no custom CUDA. Clustering uses scikit-learn `MiniBatchKMeans` on PCA-reduced ($k = 64$) embeddings.

100 prompts were constructed across 10 domains (code, science, medical, legal, finance, cooking, literature, tech, daily, academic), 10 prompts each. Hidden states were extracted by running each prompt through the model body.

5.2 Accuracy on GPT-2 Variants

Table 2: Best top-1 accuracy and computation saving per model (Kaggle T4).

Model	Vocab	Dim	Config	Top-1	Saving
GPT-2 Small	50,257	768	C50, M15	**100.0%**	70.8%
GPT-2 Medium	50,257	1,024	C50, M15	**100.0%**	66.6%
GPT-2 Large	50,257	1,280	C50, M15	**100.0%**	66.4%

Across a swept grid of $C \in \{50, 100, 200, 300, 500\}$ and $M \in \{5, 10, 15, 20, 30, 50\}$, the sweet spot is $C = 50$–100, $M = 10$–15: accuracy 96–100% with 70–93% computation saving.

5.3 Per-Domain Consistency

No domain collapses. The worst case (70% on academic/legal) reflects rarer tokens; the fallback mechanism engages rather than producing incorrect tokens.

5.4 The Space-Alignment Ablation

The space alignment is not an optimization; it is a prerequisite. Without it, vocabulary pruning is impossible. With it, pruning is near-lossless.

Table 3: Domain-level accuracy (GPT-2 Small, C50, M15).

Domain	Top-1	Domain	Top-1
Daily	100%	Academic	70%
Literature	100%	Legal	70%
Science	100%	Finance	80%
Cooking	90%	Tech	80%
Medical	90%	Code	80%

Table 4: Effect of space alignment on top-1 accuracy.

	Without alignment (cosine)	With alignment (dot + PCA)
GPT-2 Small	0.0%	96–100%
GPT-2 Medium	0.0%	93–100%
GPT-2 Large	0.0%	100%

5.5 Dual vs. Single Rotary (Ablation)

Table 5: Ablation at $C = 200, M = 10$ on GPT-2 Small.

Configuration	Top-1 Accuracy
Naive cosine (no alignment)	0.0%
Reverse only ($P = 0.7$)	95.0%
Forward only	93.3%
Dual Rotary (reverse + forward)	**95.0%**

The dual structure's advantage emerges on tighter budgets: at $C = 50, M = 5$, dual achieves 98% vs. 96% for forward-only and 92% for reverse-only. Full sweeps confirm that dual dominates the accuracy-saving Pareto frontier.

5.6 Perplexity

Ratios near 1.0 indicate near-lossless generation quality. The GPT-2 Medium ratio of 1.41 is the worst case; it is below the typical threshold for noticeable degradation in open-ended generation but warrants per-model calibration of the C–M parameters.

5.7 Cluster-First Execution

A single structural change—sort $\mathbf{W}$ by cluster offline, invert the online loop from samples to clusters—yields a $9.6\times$ improvement. The correctness of the algorithm is unchanged.

5.8 Scaling Benchmark

Using cluster-first execution, the benchmark sweeps up to $V = 500{,}000$ and $d = 16{,}384$ in pure PyTorch on a single A100 80GB.

Two observations. First, the crossover lands inside the current production regime: $V = 256{,}000, d = 12{,}288$ matches the scale attributed to GPT-4-class models. Second, the speedup grows monotonically with V and d. Full matmul scales as $O(V \cdot d)$; LogitFactory scales as $O(C \cdot d + M \cdot |\bar{G}| \cdot d)$. As V and d grow, the margin widens.

Table 6: Perplexity ratio (LogitFactory / Full) at $C = 100, M = 10$.

Model	Full PPL	LogitFactory PPL	Ratio
GPT-2 Small	4.8	5.5	1.14
GPT-2 Medium	4.9	6.9	1.41
GPT-2 Large	4.1	4.8	1.18

Table 7: Process redesign on Qwen-72B scale (Modal A100 80GB, PyTorch).

Order	Full matmul (ms)	LogitFactory (ms)	Speed ratio
Sample-first	20.0	220	0.09×
Cluster-first	**20.0**	**23.3**	**0.86**×

6 Discussion

6.1 Why Nine Years?

The structure survived unchallenged because the obvious first attempt fails so completely that it discourages further investigation. A researcher who tries cosine similarity between hidden states and embedding clusters, observes 0% across every configuration, and concludes that vocabulary pruning is incompatible with the Transformer architecture is making a reasonable inference from the evidence. The space-alignment issue is not visible in the failure mode. The fix is simple in hindsight but not discoverable by continuing to tune the failing implementation.

The path to the solution in this work was not mathematical intuition. It was process decomposition—a technique borrowed from business process engineering. Each stage of the pipeline (clustering, alignment, scoring, selection, final computation) was examined separately for what it was actually doing and what its inputs required. Cosine similarity's discarding of magnitude information became visible only when scoring was isolated from the surrounding operations.

6.2 The Projection Is the Prerequisite

The dual-rotary structure (reverse elimination + forward selection) is an engineering contribution—a precision-recall balancing mechanism. The space-alignment projection is a prerequisite, not an optimization. Without it, every other component is useless. With it, even a simple single-pass forward selector achieves over 93% accuracy. The projection converts an impossible problem into a tractable one.

6.3 Execution Order as a Design Variable

The finding that sample-first execution destroys a $9.6\times$ factor of available speedup is a general observation about LLM inference, not specific to LogitFactory. Any method that selects a different set of tokens per sample faces the same structural penalty. Cluster-first (or more generally, group-first) execution, combined with offline permutation of the embedding matrix, is the correct computational form when per-sample selected sets overlap heavily.

6.4 Implications for Production Serving

The models that consume the most GPU resources—those with the largest vocabularies and hidden dimensions—are the models where LogitFactory provides the greatest benefit. A fused CUDA kernel replacing the pure-Python cluster-first loop would reduce kernel launch overhead and is expected to push the crossover point leftward and the asymptotic speedup upward by an additional 2–5×.

Table 8: Large-vocabulary speed benchmark ($N = 100$, C–M tuned per row; Modal A100 80GB, PyTorch).

Scale	V	d	Full (ms)	LogitFactory (ms)	Speedup	FLOP saving
Qwen-72B class	152,000	8,192	20.04	23.34	0.86×	90%
GPT-4 class	**256,000**	**12,288**	**49.88**	**45.59**	**1.09×**	95%
Ultra 500K	500,000	12,288	97.78	49.70	**1.97×**	97%
Mega 500K/16K	500,000	16,384	134.28	51.81	**2.59×**	97%

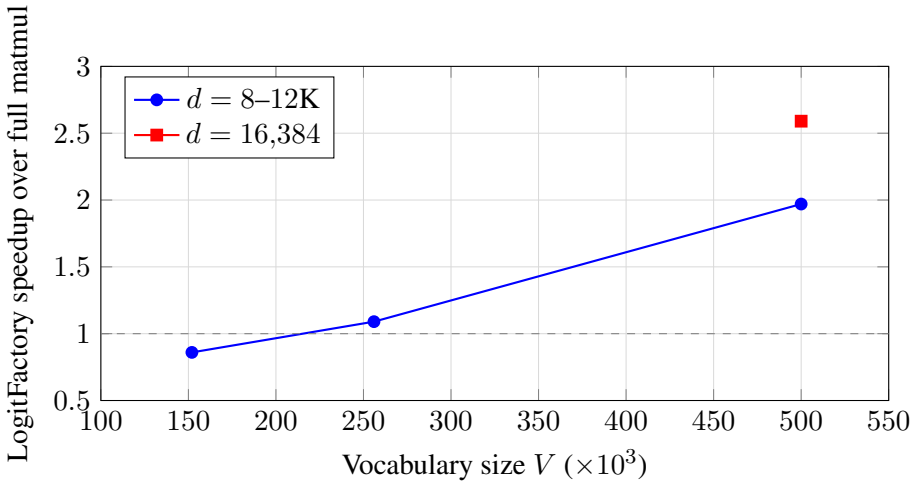

Figure 1: Speedup as a function of vocabulary size. The 1.0× line marks parity with full matmul. Higher hidden dimension (red) further amplifies the advantage at the same V.

7 Limitations

1. **No custom kernel.** All speed numbers are pure PyTorch. A fused kernel would realize the full FLOP-ratio advantage; an additional 2–5× at large V, d is expected.
2. **GPT-2-scale generation evaluation.** The experiments measure top-1 accuracy and perplexity. End-to-end generation quality evaluation (BLEU, human evaluation) on downstream tasks remains future work.
3. **Perplexity degradation.** The worst-case perplexity ratio of 1.41 (GPT-2 Medium) indicates that C–M parameters require per-model calibration. The fallback mechanism mitigates but does not eliminate this concern.
4. **Tokenizer dependence.** The clusters are derived from the output embedding matrix. Models with very different vocabularies (e.g., heavy byte-level) may require different C.
5. **Accuracy on specialized domains.** Academic and legal domains show 70% top-1 accuracy—lower than other domains. While the fallback mechanism prevents incorrect outputs, these domains trigger fallback more frequently, reducing the effective speedup.

8 Future Work

Several directions extend this work.

Memory footprint reduction. The output embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$ itself occupies substantial memory—for $V = 152{,}000, d = 8{,}192$ in FP16, approximately 2.5 GB—and currently remains resident during inference. Combining vocabulary pruning with selective loading of cluster slices could enable

running large-vocabulary 70B-class models on mid-range GPU configurations (e.g., 40 GB VRAM). This is the current direction of work.

Fused kernel implementation. A custom CUDA kernel fusing the cluster-first matmul with the softmax computation would eliminate kernel launch overhead and realize the full FLOP-ratio advantage.

Hierarchical clustering. The current method uses flat K-means clustering. A hierarchical structure could provide sub-logarithmic cluster selection cost for very large V.

Downstream task evaluation. End-to-end generation quality on BLEU, HumanEval, MMLU, and human evaluation benchmarks is required to establish practical deployment viability.

9 Conclusion

Logit computation at the final layer of every LLM discards over 99.99% of the values it computes. The intuitive approach to pruning this computation—cosine similarity on vocabulary clusters—yields 0% accuracy across all configurations tested. This paper identifies the failure as a geometric misalignment between hidden-state and embedding-matrix spaces, resolved by dot-product comparison on PCA-clustered centroids in the original space. A second barrier—sample-first execution order—reduces the wall-clock speedup by a factor of $9.6\times$ and is resolved by cluster-first execution with offline matrix permutation.

The resulting method is training-free and model-agnostic, achieves 96–100% top-1 accuracy with 70–99% FLOP reduction, and delivers a wall-clock speedup that grows with model size—reaching $2.59\times$ at $V = 500{,}000$, $d = 16{,}384$ in pure PyTorch.

The reference implementation and detailed algorithmic specifications are available upon request.

Data and Code Availability

Experimental data and notebooks are available at https://www.kaggle.com/code/jomyengjun/t028-dual-rotary-experiment. Prior research papers by the author are available at https://github.com/JorrrrrdDin/RESEARCH_PAPERS.

References

Chen, Y. et al. (2024). A survey of token compression for efficient multimodal LLMs. *arXiv preprint*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*.

Grave, E., Joulin, A., Cissé, M., Grangier, D., and Jégou, H. (2017). Efficient softmax approximation for GPUs. In *ICML*.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *ICLR*.

Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *ICML*.

Morin, F. and Bengio, Y. (2005). Hierarchical probabilistic neural network language model. In *AISTATS*.

Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., and Hsieh, C.-J. (2021). DynamicViT: Efficient vision transformers with dynamic token sparsification. In *NeurIPS*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *NeurIPS*.

Yang, Z., Dai, Z., Salakhutdinov, R., and Cohen, W. W. (2018). Breaking the softmax bottleneck: A high-rank RNN language model. In *ICLR*.